



COS30019: Introduction to Artificial Intelligence

Logical Agents & Knowledge Representation

Propositional Logic

1



Outline

- Propositional logic
 - ☐ Syntax
 - ☐ Semantics
- Truth table based inference
- Proof methods
 - ☐ Resolution
 - ☐ Forward chaining
 - ☐ Backward chaining



2

Propositional logic: Syntax



- Propositional logic is the simplest logic – illustrates basic ideas
- The **proposition symbols** S, P_1, P_2 etc are sentences
 - If S is a sentence, $\neg S$ is a sentence (**negation**)
 - If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence (**conjunction**)
 - If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence (**disjunction**)
 - If S_1 and S_2 are sentences, $S_1 \Rightarrow S_2$ is a sentence (**implication**)
 - If S_1 and S_2 are sentences, $S_1 \Leftrightarrow S_2$ is a sentence (**biconditional**)



3

Example



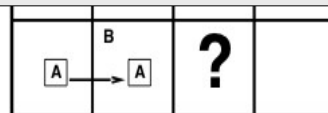
- $P_{1,2} = \{T/F\}$ (there is/isn't a pit in cell [1,2])
- $P_{2,2} = \{T/F\}$ (there is/isn't a pit in cell [2,2])
- $P_{3,1} = \{T/F\}$ (there is/isn't a pit in cell [3,1])

Situation after detecting nothing in [1,1],

$P_{1,2}, P_{2,2}, P_{3,1}$ are proposition symbols

Consider possible models for ?s
assuming only pits

3 Boolean choices $\Rightarrow$ 8 possible models



4

Example

- Let $KB = \{$

- $R : It rains$
- $U : I have an umbrella$
- $W : I get wet$

U, R, W are proposition symbols

The sentence $\neg U$ means "*I have NO umbrella*"

When

The sentence $R \wedge \neg U \Rightarrow W$ means 'IF "*it rains*" AND "*I have no umbrella*" THEN "*I get wet*"'

Do

$KB = \{$
 $R,$
 $R \wedge \neg U \Rightarrow W,$
 $\neg U \}$

SWINBURNE

CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

5

Propositional logic: Semantics

- Each model specifies true/false for each proposition symbol
- E.g. $P_{1,2}$ $P_{2,2}$ $P_{3,1}$
 false true false

With these 3 symbols, 8 possible models, can be enumerated automatically.

- Rules for evaluating
- $\neg S$ is tr
- $S_1 \wedge S_2$ is tr
- $S_1 \vee S_2$ is tr
- $S_1 \Rightarrow S_2$ is tr
- i.e., is fa
- $S_1 \Leftrightarrow S_2$ is tr

- Simple recursive pr
- $\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1})$

Wumpus world model

	P12	P22	P31
Model 1	F	F	F
Model 2	F	F	T
Model 3	F	T	F
Model 4	F	T	T
Model 5	T	F	F
Model 6	T	F	T
Model 7	T	T	F
Model 8	T	T	T

SWINBURNE

CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

6

Truth tables for connectives



P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Wumpus world sentences



Let $P_{i,j}$ be true if there is a pit in $[i, j]$.

Let $B_{i,j}$ be true if there is a breeze in $[i, j]$.

$$\alpha_1 = \neg P_{1,1}$$

$$\alpha_2 = \neg B_{1,1}$$

$$\alpha_3 = B_{2,1}$$

■ "Pits cause breezes in adjacent squares"

$$\square \alpha_4 = B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$\square \alpha_5 = B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

■ $KB = \{ \alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5 \}.$

Truth tables for inference



$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	KB	α_1
false	false	false	false	false	false	false	false	true
false	false	false	false	false	false	true	false	true
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	false	true
false	true	false	false	false	false	true	true	true
false	true	false	false	false	true	false	true	true
false	true	false	false	false	true	true	true	true
false	true	false	false	true	false	false	false	true
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false	false



9

Inference by enumeration



- Depth-first enumeration of all models is sound and complete

■

```

function TT-ENTAILS?( $KB, \alpha$ ) returns true or false
  symbols  $\leftarrow$  a list of the proposition symbols in  $KB$  and  $\alpha$ 
  return TT-CHECK-ALL( $KB, \alpha, symbols, []$ )

function TT-CHECK-ALL( $KB, \alpha, symbols, model$ ) returns true or false
  if EMPTY?( $symbols$ ) then
    if PL-TRUE?( $KB, model$ ) then return PL-TRUE?( $\alpha, model$ )
    else return true
  else do
     $P \leftarrow$  FIRST( $symbols$ );  $rest \leftarrow$  REST( $symbols$ )
    return TT-CHECK-ALL( $KB, \alpha, rest, EXTEND(P, true, model)$ ) and
           TT-CHECK-ALL( $KB, \alpha, rest, EXTEND(P, false, model)$ )
    
```

- For n symbols, time complexity is $O(2^n)$, space complexity is $O(n)$



10

Logical equivalence



- Two sentences are **logically equivalent** iff true in same models:

$\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$

$$\begin{aligned}
 (\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
 (\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
 ((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
 ((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
 \neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
 (\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
 (\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
 (\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
 \neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{de Morgan} \\
 \neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{de Morgan} \\
 (\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
 (\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
 \end{aligned}$$



11

Validity and satisfiability



- A sentence is **valid** if it is true in **all** models,
e.g., *True*, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$
- Validity is connected to inference via the **Deduction Theorem**:
 $KB \models \alpha$ if and only if $(KB \Rightarrow \alpha)$ is valid
- A sentence is **satisfiable** if it is true in **some** model
e.g., $A \vee B$, C
- A sentence is **unsatisfiable** if it is true in **no** models
e.g., $A \wedge \neg A$
- Satisfiability is connected to inference via the following:
 $KB \models \alpha$ if and only if $(KB \wedge \neg\alpha)$ is unsatisfiable



12

Proof methods



■ Proof methods divide into (roughly) two kinds:

- **Application of inference rules**
 - Legitimate (sound) generation of new sentences from old
 - **Proof** = a sequence of inference rule applications
Can use inference rules as operators in a standard search algorithm
 - Typically require transformation of sentences into a **normal form**
- **Model checking**
 - truth table enumeration (always exponential in n)
 - If implemented correctly, give you're the **standard entailment** $\models$
 - improved backtracking, e.g., Davis--Putnam-Logemann-Loveland (DPLL)
 - heuristic search in model space (sound but incomplete)
e.g., min-conflicts-like hill-climbing algorithms



13

Resolution



Conjunctive Normal Form (CNF)

conjunction of disjunctions of literals
clauses

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

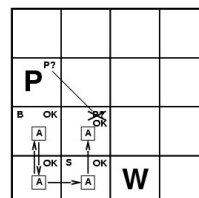
■ Resolution inference rule (for CNF):

$$\frac{l_i \vee \dots \vee l_k, \quad m_1 \vee \dots \vee m_n}{l_i \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where l_i and m_j are complementary literals.

$$\text{E.g., } \frac{P_{1,3} \vee P_{2,2}, \quad \neg P_{2,2}}{P_{1,3}}$$

- Resolution is sound and complete for propositional logic



14

Resolution algorithm



- Proof by contradiction, i.e., show $KB \wedge \neg \alpha$ unsatisfiable
-

```

function PL-RESOLUTION( $KB, \alpha$ ) returns true or false
  clauses  $\leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg \alpha$ 
  new  $\leftarrow \{ \}$ 
  loop do
    for each  $C_i, C_j$  in clauses do
      resolvents  $\leftarrow$  PL-RESOLVE( $C_i, C_j$ )
      if resolvents contains the empty clause then return true
      new  $\leftarrow$  new  $\cup$  resolvents
    if new  $\subseteq$  clauses then return false
    clauses  $\leftarrow$  clauses  $\cup$  new
  
```

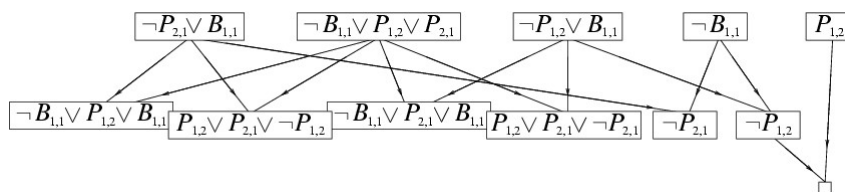


15

Resolution example



- $KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1} \quad \alpha = \neg P_{1,2}$
-



16

Forward and backward chaining



- **Horn Form** (restricted)

KB = **conjunction** of **Horn clauses**

- Horn clause =

- proposition symbol; or

- (conjunction of symbols) $\Rightarrow$ symbol

- E.g., $C \wedge (B \Rightarrow A) \wedge (C \wedge D \Rightarrow B)$

- **Modus Ponens** (for Horn Form): complete for Horn KBs

$$\frac{\alpha_1, \dots, \alpha_n, \quad \alpha_1 \wedge \dots \wedge \alpha_n \Rightarrow \beta}{\beta}$$

- Can be used with **forward chaining** or **backward chaining**.
- These algorithms are very natural and run in **linear** time



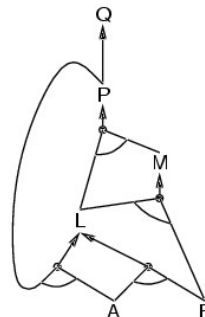
17

Forward chaining



- Idea: fire any rule whose premises are satisfied in the KB,
 - add its conclusion to the KB, until query is found

$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

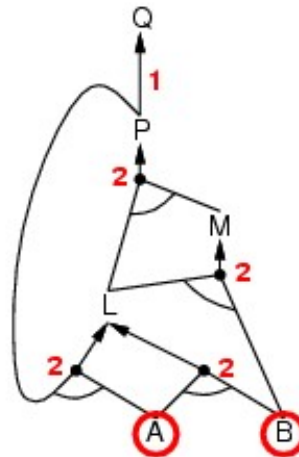


18

Forward chaining example



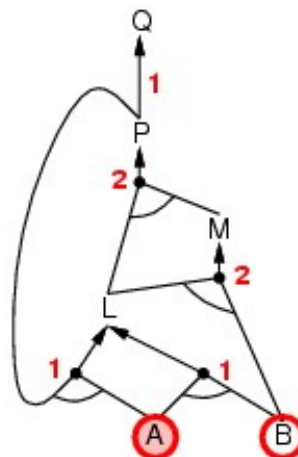
$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



Forward chaining example



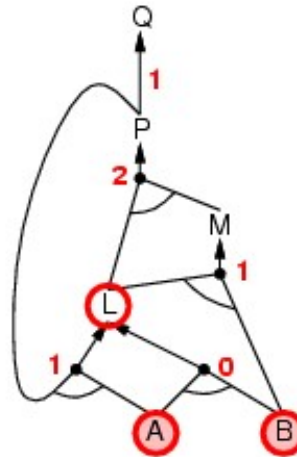
$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



Forward chaining example



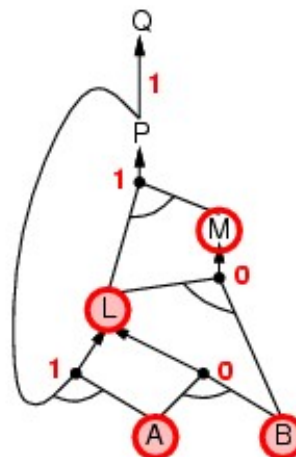
$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



Forward chaining example



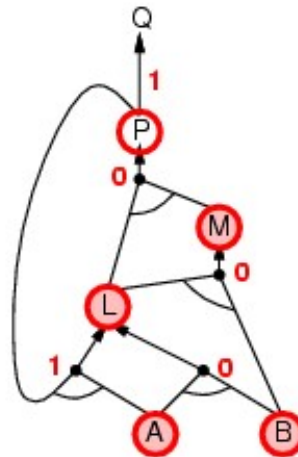
$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



Forward chaining example



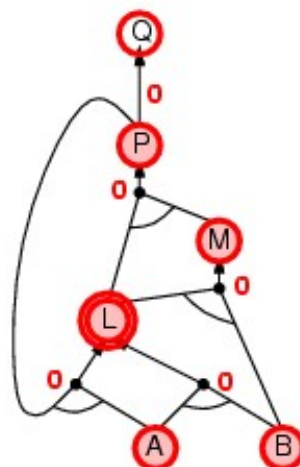
$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



Forward chaining example



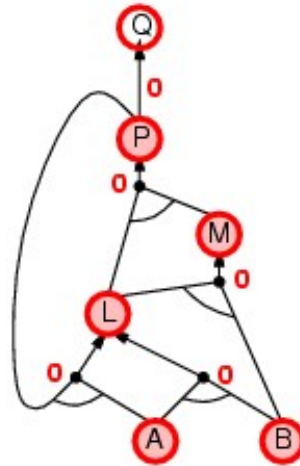
$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



Forward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

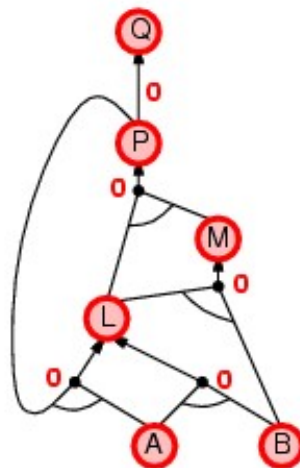


25

Forward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



26

Forward chaining algorithm



```
function PL-FC-ENTAILS?(KB, q) returns true or false
  local variables: count, a table, indexed by clause, initially the number of premises
                  inferred, a table, indexed by symbol, each entry initially false
                  agenda, a list of symbols, initially the symbols known to be true

  while agenda is not empty do
    p ← POP(agenda)
    unless inferred[p] do
      inferred[p] ← true
      for each Horn clause c in whose premise p appears do
        decrement count[c]
        if count[c] = 0 then do
          if HEAD[c] = q then return true
          PUSH(HEAD[c], agenda)
  return false
```

- Forward chaining is sound and complete for Horn KB
-



27

Backward chaining



Idea: work backwards from the query q :

to prove q by BC,

check if q is known already, or

prove by BC all premises of some rule concluding q

Avoid loops: check if new subgoal is already on the goal stack

Avoid repeated work: check if new subgoal

1. has already been proved true, or
2. has already failed

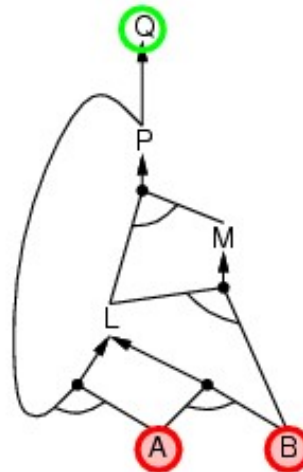


28

Backward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

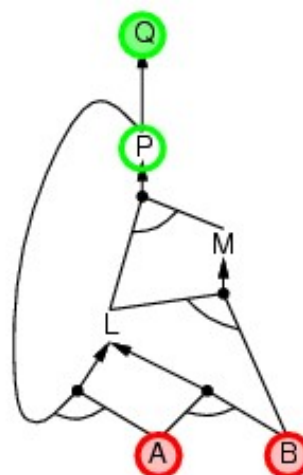


29

Backward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

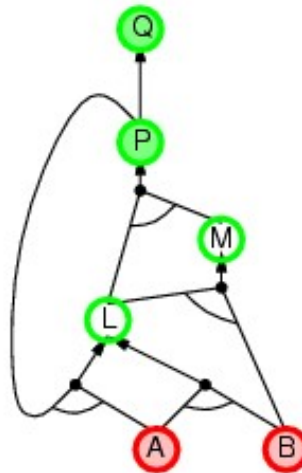


30

Backward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

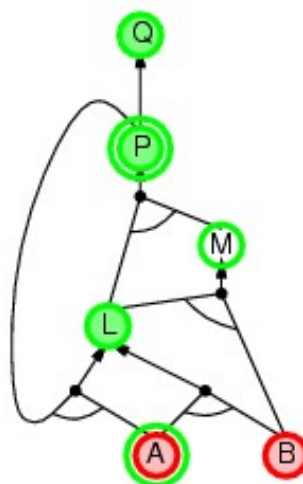


31

Backward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

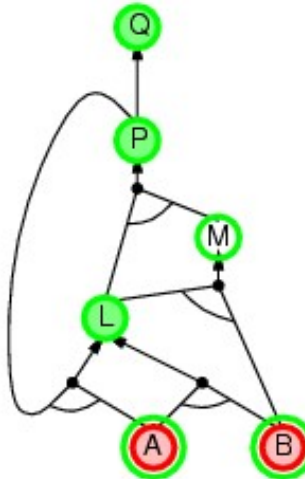


32

Backward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

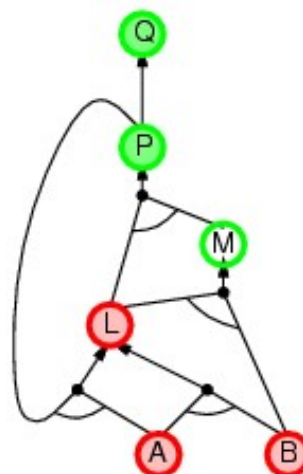


33

Backward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

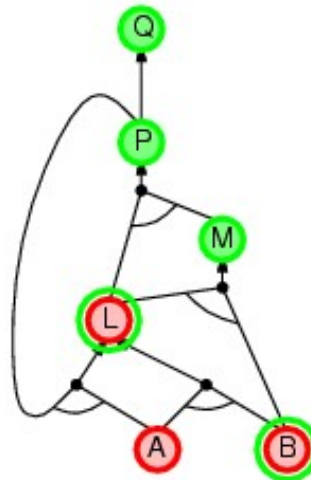


34

Backward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

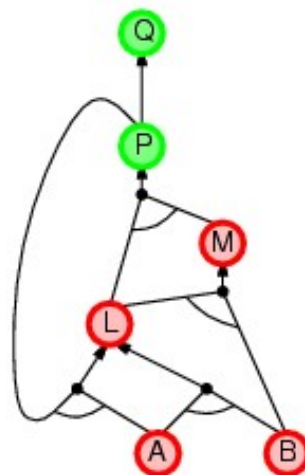


35

Backward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

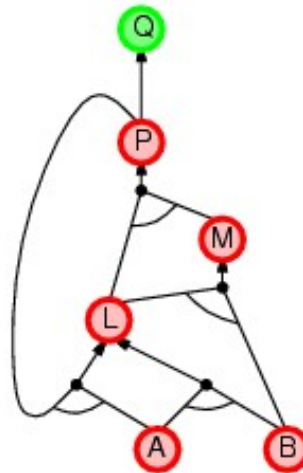


36

Backward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

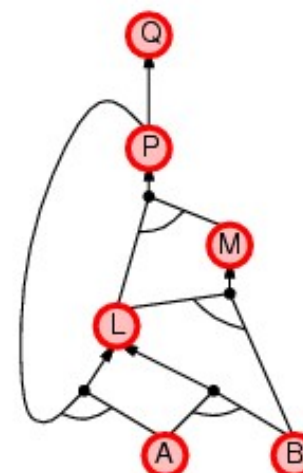


37

Backward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



38

Forward vs. backward chaining



- FC is **data-driven**, automatic, unconscious processing,
 - e.g., object recognition, routine decisions
- May do lots of work that is irrelevant to the goal
- BC is **goal-driven**, appropriate for problem-solving,
 - e.g., Where are my keys? How do I get into a PhD program?
- Complexity of BC can be **much less** than linear in size of KB



39

Summary



- Logical agents apply **inference** to a **knowledge base** to derive new information and make decisions
- Basic concepts of logic:
 - **syntax**: formal structure of **sentences**
 - **semantics**: **truth** of sentences wrt **models**
 - **entailment**: necessary truth of one sentence given another
 - **inference**: deriving sentences from other sentences
 - **soundness**: derivations produce only entailed sentences
 - **completeness**: derivations can produce all entailed sentences
- Wumpus world requires the ability to represent partial and negated information, reason by cases, etc.
- Resolution is complete for propositional logic
Forward, backward chaining are linear-time, complete for Horn clauses
- Propositional logic lacks expressive power
 - First order logic (FOL) addresses this issue



40